1.3) Data Type.

- **Input:**
  - myfirst_int=43
  - myfirst_float=27.4
  - my_sum=myfirst_int+myfirst_float
  - print(my_sum)
  - print(type(my_sum))
  - my_int=65
  - my_str='53'
  - my_plus=my_int+int(my_str)
  - print(my_plus)
  - print(type(my_plus))
- **Output:**

```
70.4
<class 'float'>
118
<class 'int'>
```

1.4) Aim: To Write A Program For Swapping Of Two Numbers With And Without Creating Third Variable.

- **With Creating Third Variable:**
- **Input:**
  - a=15
  - b=20
  - temp=a #15
  - a=b
  - b=temp
  - print('A:',a)
  - print('B:',b)
- **Output:**

```
A: 20
B: 15
```

- **Without Creating Third Variable:**
- **Input:**
  - X=20
  - Y=30
  - X,Y=Y,X
  - print('X:',X)
  - print('Y:',Y)
- **Output:**

```
X: 30
Y: 20
```

1.5) Aim: To Write A Program For Swapping Of Two Numbers By Using Input Function.

- **By Using Third Variables:**
- **Input:**
  - #swapping two numbers
  - a=int(input('Enter first no:'))
  - b=int(input('Enter second no:'))
  - #swapping variables
  - c=a
  - a=b
  - b=c
  - #display swap numbers
  - print('There is your swap number:')
  - print(a)
  - print(b)
- **Output:**

```
===== RESTART: C:\Users\User\Ap
Enter first no:100
Enter second no:150
There is your swap number:
150
100
```

- **By without using third variable:**
- **Input:**
  - a=int(input('Enter first no:'))
  - b=int(input('Enter second no:'))
  - #swapping variable without third variable
  - #another easy way a,b=b,c
  - a=a+b
  - b=a-b
  - a=a-b
  - #display swap number
  - print('There is your swap number:')
  - print(a)
  - print(b)
- **Output:**

```
Enter first no:15
Enter second no:30
There is your swap number:
30
15
```

1.6) Operators in Python.

1) Arithmetic Operator:

- **Input:**
  - x=10
  - y=5
  - print(x+y)
  - print(x-y)
  - print(x*y)
  - print(x/y)
  - print(x%y)
  - print(x**y)
- **Output:**

```
===== RESTART: C:\Users
15
5
50
2.0
0
100000
>
```

2) Assignment operator:

Input:

- x=10
- x+=5
- print(x)
- y=9
- y-=3
- print(y)
- z=11
- z*=3
- print(z)
- e=10
- e%=4
- print(e)
- a=8
- a//=4
- print(a)
- r=5
- r/=2
- print(r)

- **Output:**

```
15
6
33
2
2
2.5
```

3) Comparison operator:

- **Input:**
  - print('equal:',6==3)
  - print('not equal:',6!=3)
  - print('greater than:',6>3)
  - print('less than:',6<3)
  - print('greater than or equal to:',6>=3)
  - print('less than or equal to:',6<=3)
- **Output:**

```
equal: False
not equal: True
greater than: True
less than: False
greater than or equal to: True
less than or equal to: False
```

4) Logical Operator:

- **Input:**
  - a,b,c=True,False,True
  - if a and c:
  - print('both a and c are True(AND opreation).')
  - if b or c:
  - print('either b or c is Ture(OR opreation).')
  - if not b:
  - print('b is False(NOT opreation).')
- **Output:**

```
both a and c are True(AND opreation).
either b or c is Ture(OR opreation).
b is False(NOT opreation).
```

5) Identity Operator:

- **Input:**
  - x=['apple','iphone']
  - y=['apple','iphone']
  - z=x
  - print(x is z)
  - print(x is not y)
  - print(x==y)
- **Output:**

```
True
True
True
```

6) **Bitwise Operator:**

- **Input:**
  - a,b,c=True,False,True
  - if a & c:
  - print('both a and c are True(& condition).')
  - if b|c:
  - print('either b or c is True(| condition).')
- **Output:**

```
both a and c are True(& condition).
either b or c is True(| condition).
```

7) **Membership Operator:**

- **Input:**
  - number_list=[1,2,3,4,5,6,7,8,9]
  - print(1 in number_list)
  - print(10 not in number_list)
  - print(6 in number_list)
- **Output:**

```
True
True
True
```

1.7) Dictionary:

- **Input: Create Dictionary:**
  - thedict={'brand':'toyota','model':'mustang','year':2014}
  - print(thedict)
- **Output:**

```
===== RESTART: C:\Users\User\AppData\Local\Programs\Py
{'brand': 'toyota', 'model': 'mustang', 'year': 2014}
```

- **Input: Access A Particular Elements In Dictionary:**
  - thedict={'brand':'toyota','model':'mustang','year':2014}
  - print(thedict)
  - x=thedict['brand']
- **Output:**

```
{'brand':
toyota
```

- **1st Way Input: Update A Particular Elements Dictionary:**
  - # 1st way dict
  - car={'brand':'toyota','model':'mustang','year':2014}
  - x=car.values()
  - print(x)
  - car['year']=2019
  - print(x)
- **Output:**

```
dict_values(['toyota', 'mustang', 2014])
dict_values(['toyota', 'mustang', 2019])
```

- **2nd Way Input:**
  - # 2nd way dict
  - thisdict={'brand':'toyota','model':'mustang','year':2014}
  - thisdict.update({'year':2019})
  - print(thisdict)
- **Output:**

```
{'brand': 'toyota', 'model': 'mustang', 'year': 2019}
```

- **Input: Adding Items.**

- thedict={'brand':'toyota','model':'mustang','year':2014}
- thedict['color']='red'
- print(thedict)

➢ **Output:**

```
{'brand': 'toyota', 'model': 'mustang', 'year': 2014, 'color': 'red'}
```

➢ **Input: Remove Items.**

- thedict={'brand':'toyota','model':'mustang','year':2014}
- thedict.pop('model')
- print(thedict)

➢ **Output:**

```
{'brand': 'toyota', 'year': 2014}
```

➢ **Input: Clear All Elements.**

- thedict={'brand':'toyota','model':'mustang','year':2014}
- thedict.clear()
- print(thedict)

➢ **Output:**

```
{}
```

1.7) Set:

➢ **Input: Create Set.**

- thisset={'apply','mango','cherry'}
- print(thisset)

➢ **Output:**

```
{'cherry', 'mango', 'apply'}
```

➢ **Input: Access Items.**

- thisset={'apply','mango','cherry'}
- for x in thisset:
- print(x)

➢ **output:**

```
cherry
mango
apply
```

- **Input: Add Items.**
  - thisset={'apply','mango','cherry'}
  - thisset.add('orange')
  - print(thisset)
- **Output:**

```
{'cherry', 'mango', 'apply', 'orange'}
```

- **Input: Update Items.**
  - thisset={'apply','mango','cherry'}
  - theset={'pineapply','banana','papaya'}
  - thisset.update(theset)
  - print(thisset)
- **Output:**

```
{'papaya', 'pineapply', 'cherry', 'mango', 'apply', 'banana'}
```

- **Input: Remove Items.**
  - thisset={'apply','mango','cherry'}
  - thisset.remove('cherry')
  - print(thisset)
- **Output:**

```
{'mango', 'apply'}
```

- **Input: Clear All Elements.**
  - thisset={'apply','mango','cherry'}
  - thisset.clear()
  - print(thisset)
- **Output:**

```
set()
```

1.2) Comments, Docstring And Demonstrates.

- **Input:**
  - #this is single line comments
- **Output:**

```
# this is single line comments
```

- **Input:**
  - # this is multiline comments
- **Output:**

```
''' this is multiline comments'''
```

- **Input:**
  - def my_function():
  - '''demonstrates triple double quotes
  - docstrings and does nothing really.'''
  - print('using__doc__:')
  - print(my_function.__doc__)
- **Output:**

```
===== RESTART: C:\Users\User\AppData\
using__doc__:
demonstrates triple double quotes
docstrings and does nothing really.
```